

Exploring Large Language Models for Generating RDF Triples from NL texts: Findings in the e-commerce domain

André Gomes Regino   [DIMEC, Center for Information Technology Renato Archer | aregino@cti.gov.br]

Julio Cesar dos Reis  [Institute of Computing, University of Campinas, Unicamp | jreis@ic.unicamp.br]

Received: DD Month YYYY • Accepted: DD Month YYYY • Published: DD Month YYYY

Abstract. Knowledge Graphs (KGs) have been used to represent information in a structured way, preserving semantics and facilitating the refined retrieval and understanding of complex information through semantic queries. By expressing relationships and hierarchies among entities, KGs play a fundamental role in organizing and connecting heterogeneous data. In the context of e-commerce, KGs represent knowledge about products, stores, user behavior, and purchases. This knowledge originates from several sources, including natural-language (NL) texts from customers, partners, and retailers. Deep Learning techniques have been used in state-of-the-art approaches to process such texts, including the adoption of Large Language Models. In this article, we propose a novel method that leverages Large Language Models (LLMs) to generate RDF triples from natural-language questions in e-commerce. Our proposal unfolds three key steps: preprocessing with ontology elements, LLM-based triple generation, and integration with the existing KG. We combined the definitions present in the ontology with the instance examples of the KG and the text generation power of LLMs to generate new RDF triples. The experimental evaluations, conducted on compatibility questions from multiple stores in the automobile domain in Portuguese, demonstrate the effectiveness of the proposed approach in generating semantically coherent triples aligned with predefined ontology definitions. Our contribution emphasizes the intersection of NL understanding, knowledge representation, and e-commerce, paving the way for advanced information retrieval and enrichment applications.

Keywords: E-commerce, Semantic Web, Natural Language Processing, Large Language Models

1 Introduction

In e-commerce, exchanging information between humans and machines has become increasingly mandatory. Consumers frequently seek information about products, services, and specifications through natural language queries, forming the basis of their interactions with online platforms. Consequently, bridging the gap between human-friendly natural language and machine-understandable data structures has become challenging in this domain [19].

The e-commerce sector, characterized by its vast, dynamic product range, is inundated with customer questions, ranging from product compatibility to user preferences. One common scenario involves compatibility questions, where customers inquire about the suitability of specific products within a particular context, such as, “Is this smartphone case compatible with my Samsung S24?” These questions, often formulated in natural language, require precise and structured responses that can be integrated into the existing knowledge structure of e-commerce platforms. Examples of computational knowledge structures are ontologies and Knowledge Graphs (KGs) [7].

Using ontologies and KGs in e-commerce has been a promising approach to structuring product-related information [21]. Ontologies provide a formal representation of knowledge comprising classes, properties, and relationships among entities [8], facilitating a structured, standardized way to represent domain-specific information. The main component of these structures is the RDF (Resource Description Framework), which is a graph of triples organized by subject, predicate, and object [15].

The structured format of KGs facilitates queries and enables stakeholders to extract valuable insights. By appending information to KGs, e-commerce platforms enhance their capacity to derive relationships, identify patterns, and offer personalized recommendations, creating an environment of informed decision-making for businesses and customers. The graph-structured nature of KGs might help to transform unstructured text into a valuable resource, empowering e-commerce stakeholders with a broad understanding of their domain. However, generating RDF triples to augment KGs is arduous and challenging.

The main challenge lies in automating the generation of RDF triples from NL questions and in connecting human-generated queries to an existing structured ontology. E-commerce texts often contain territory-specific expressions, ambiguous terms, and context-dependent references, making it difficult to extract precise, structured information. The lack of patterns in product categorization, the diversity of user queries, and the evolution of language trends further compound the challenge. Additionally, e-commerce systems frequently handle dynamic, vast datasets, requiring an adaptable approach to handle varying levels of detail and context. The scientific problem in our investigation is to develop a method that accurately interprets the diverse and dynamic NL expressions in e-commerce texts and aligns the generated triples with existing ontological structures, ensuring semantic coherence and relevance.

Our investigation addresses the need for a methodological solution that leverages the power of Large Language Models (LLMs) to bridge the research gap described. In particular, we propose an original approach to automatically generate

RDF triples from NL questions in e-commerce, enabling precise, context-aware responses to user queries. Our research addresses the “how” and the “how-to-link” aspects, ensuring that identified triples are adequately integrated into existing ontologies, and adhere to defined classes and properties.

In summary, our key research question in this investigation is: How can LLMs be used to generate triples from ontology specifications?

This investigation represents one of the modules of a larger framework called QART [18]. This framework generates triples from NL texts. The first stage of QART, described in detail in Regino *et al.* [18], described how we transform e-commerce texts into summarized sentences that express the primary meaning of the original sentences. These summarized sentences are used to create triples. This transformation is the main subject of our present study and the summarized sentences are the input of the method described in Section 3.

This problem was identified in the context of a Brazilian startup offering artificial intelligence solutions for e-commerce, GoBots. This startup constructed a KG that is populated daily. For the startup’s business model, the more questions the developed KG automatically answers, the better: the seller answers fewer repetitive questions, fewer human errors, and responds to the customer more quickly. Implementing our method would make it possible to insert new data into the KG from a broader range of intents (goals or actions that users intend to perform when interacting with the e-commerce, like purchase, cart addition, compatibility), entities (relevant details or parameters that fulfill the user’s intent, like product names, categories, brands), and domains created in natural language, consequently increasing the rate of questions answered by the KG. In particular, we address NL written in Portuguese.

The two evaluations presented in this study share a common goal: assessing LLM-generated RDF triples for e-commerce Q&A. They differ in scope and focus. The first evaluation (cf. Section 4) assesses and measures the accuracy and completeness of the generated triples, examining how variations in input prompts and model selection affect the representation of product relationships. It aims to understand the effectiveness of LLMs in transforming NL into structured knowledge, without considering factors such as dataset composition and text intent. The second evaluation (cf. Section 5) takes a broader perspective by investigating multiple influencing factors in triple generation, including domain diversity, intent categorization, different prompt strategies, and LLMs. Comparing datasets with varying levels of domain specificity and testing models with different training characteristics provides a systematic analysis of how these elements shape the generated triples. These evaluations offer complementary insights into the challenges and potential of using LLMs for KG construction in e-commerce.

The remaining of this article is organized as follows: Section 2 describes studies related to RDF triple generation and natural language transforming; Section 3 describes our proposal and method; Section 4 presents how we evaluate our method using compatibility type of questions in the e-commerce scenario and Section 5 presents how we evaluate multi intents, domains and LLMs hyperparameters; Section 6 discusses our findings and open future research challenges;

and Section 7 draws conclusion remarks.

2 Related Work

This section reviews and discusses the existing literature on integrating LLMs and KGs. We identify studies related to our domain (e-commerce) and techniques from other domains connected to our present study.

Seq2RDF [14] uses the Transformer architecture to construct triples. Seq2RDF was one of the first Transformers applications to build KGs in 2018. The methodology starts with inputting training data (DBpedia), followed by a set of encoder/decoder layers in a neural network (Transformers architecture), outputting a set of RDF triples. Different from our method, Seq2RDF is multidomain. The authors evaluate it quantitatively by comparing it with a gold standard. We evaluated our method using e-commerce-related gold standards. Even though Seq2RDF is considered efficient and straightforward [14] for the triple generation task, it can only generate a single triple per sentence. We could not apply Seq2RDF to our problem and context of generating triples in e-commerce because generating knowledge demands the creation of more than one triple, regardless of how many intentions (compatibility, availability, product specification) and how many entities (item being sold, store, buyer’s item) are present in the sentence written in natural language.

Unlike Seq2RDF, Rossanez *et al.* [20] developed a specialized approach for generating KGs in the biomedical domain named KGen. Their approach incorporated ontology definitions to append biomedical RDF triples to an existing KG. The methodology was structured around four key stages: preprocessing, triple extraction, linking to ontologies, and ultimately creating the KG. Their primary objective was to extract insights from biomedical texts, align them with existing ontologies, and synthesize a KG. The evaluations encompassed both quantitative and qualitative analyses. However, challenges arose from the methodology’s lack of full automation, as the method proposed by Rossanez *et al.* is semi-automatic.

Distinct from Rossanez *et al.* [20], Balabin *et al.* [4] studied a transformer trained on biomedical text for creating KGs, demonstrating adaptability to other domains. Evaluations encompass quantitative analyses and comparisons with a gold standard. The solution’s strengths lie in its adaptability across domains and its superior effectiveness compared to reference models such as BioBert [12] and node2vec [9], particularly with a large number of classes. However, it has limitations, such as being tested exclusively in the biomedical context and requiring longer training times than other models.

Li *et al.* [13] employed a method named “AliMe KG” involving phrase mining, named entity recognition, relation extraction, and knowledge fusion. The authors delineated a semi-automated process for knowledge acquisition and validation, integrating human annotation and feedback cycles to augment the precision and completeness of Knowledge Graphs. In the e-commerce domain, which aligns with our focus, the study highlighted real-world applications of the AliMe KG in pre-sales conversations. The evaluation of “Ali-

iMe KG” encompassed both quantitative and qualitative assessments. Open challenges included extending “AliMe KG” coverage in the Alibaba e-commerce platform and exploring its application across various domains beyond e-commerce.

In summary, the reviewed related work demonstrates a rich landscape of research exploring the integration of LLMs and KGs. Our investigation contributes to this body of knowledge by addressing specific challenges related to triple generation and knowledge representation in the e-commerce domain. To the best of our knowledge, there is no evidence of a study framework that is both designed and fully implemented for producing RDF triples from e-commerce natural language texts and appending them to an existing KG using LLMs.

Table 1 summarizes the characteristics of each revised study described in this section. The last line presents our method. Our proposal contributes to the state of the art in transforming summarized texts into triples, part of a larger framework, QART [18].

3 Generating RDF Triples from NL texts

This section describes our solution for generating RDF triples from NL text in e-commerce. Our solution transforms NL-written text into triples. We first describe our proposal (cf. Subsection 3.2) and then present a running example to illustrate our solution (cf. Subsection 3.3).

3.1 Definitions and Formalizations

KGs and ontologies provide a framework for systematically organizing and interconnecting information, ensuring consistency, interoperability, and inferential capabilities. These concepts are necessary in multiple real-world scenarios, including search engines, recommendation systems, and automated reasoning in specialized fields such as healthcare and e-commerce. Below, we contextualize and formalize the key components of these structures and provide some examples.

The ontologies, also called T-Box, have been studied for years to improve data representation in graph form [11].

Definition 3.1 (Ontology ($\mathcal{O}$)). A shared and formal conceptualization and representation of knowledge [11]. An ontology $\mathcal{O}$ is formed by the components (Cl , Pr , Re), where Cl represents a set of classes, Pr is the set of properties, and Re is the set of relationships.

Definition 3.2 (Class (Cl_i)). Classes are the set of object types defined in an ontology [Gruber]. They represent a category or type within the ontology, representing a set of entities with common characteristics. Formally, Cl_i is an element of the set of classes Cl in the ontology.

Definition 3.3 (Property (Pr_i)). The properties are attributes of the objects defined in the classes [Gruber]. They represent characteristics or attributes of entities (real-world concepts and objects) in the ontology. Pr_i is an element of the ontology’s set of properties Pr .

Definition 3.4 (Knowledge Graph ($\mathcal{KG}$)). A graph-based representation of data is constructed by linking RDF triples according to the ontology’s structure. RDF Triples follow the RDF model, a standard for describing named entities, properties, and values [15]. A Knowledge Graph $\mathcal{KG}$ is a connected directed graph formed by $\mathcal{V}$ vertices (the entities, objects of Cl) and their directed edges $\mathcal{E}$ (the connections between entities, the relationships Re).

KGs serve as invaluable assets across diverse domains, ranging from applications related to improving search engine outcomes [28] to those targeting the enhancement of the results of artificial intelligence models and applications [23]. These structured knowledge repositories find applications in diverse tasks, including but not limited to information retrieval [2], query and answering questions [29], and data integration [24; 6].

They are also present in a wide range of applications. For example, in e-commerce, KGs help enhance user experiences by providing personalized recommendations and ensuring product compatibility through semantic relationships extracted from textual descriptions [13]. In healthcare, KGs constructed from scientific articles and electronic health records facilitate the identification of clinical patterns and support evidence-based decision-making [26]. The automotive industry’s KGs derived from technical manuals and specifications enable precise queries regarding part compatibility and vehicle maintenance [25].

One example of a widely-known and freely available KG is DBpedia [3]. It is generated by transforming textual data from Wikipedia infoboxes into RDF triples.

Definition 3.5 (Triple (T_i)). A statement in the form of a subject-predicate-object, representing a relationship between entities in the $\mathcal{KG}$. Formally, T_i is the triple (s, p, o) where s is the subject entity, p is the predicate, and o is the object entity. We define the set of triples T_i as $\mathcal{T}$.

Triples are used in structured knowledge representation because they explicitly capture semantic relationships, allowing machines to interpret and reason over data. Unlike traditional tabular representations, which fragment information across multiple tables and rely on foreign key relationships, triples provide a direct way to model connections between concepts.

Triples are often represented using the Resource Description Framework (RDF), a W3C-endorsed model¹ that encodes information in a machine-readable format. RDF assigns Uniform Resource Identifiers (URIs) to subjects and predicates. This way, entities and relationships are uniquely identifiable across datasets, enabling interoperability and standardization. Additionally, RDF graphs can be queried using SPARQL², a specialized query language that allows expressive and efficient retrieval of structured information.

Definition 3.6 (Prompt ($\mathcal{P}$)). A structured textual input is provided to a language model to guide its response generation. In our context, a prompt $\mathcal{P}$ is composed of: (i) the instruction part $P_{instructions}$, which defines the task to be

¹<https://www.w3.org/TR/rdf12-n-triples/>

²<https://www.w3.org/TR/sparql11-query/>

Table 1. Summary of state-of-art characteristics. The lines show each related work described in Section 2. Each row describes a characteristic of the related work: the LLM usage in the method; the usage of natural language texts as input; the generation of Knowledge Graphs as output; the appending of the resulting triples to an existing KG and the application in the e-commerce domain.

	Use LLMs	NL texts	Generate KGs	Append to Existing KG	E-commerce domain
Seq2RDF [14]		✓	✓	✓	
KGen [20]		✓	✓	✓	
STonKGs [4]		✓	✓		
AliMe KG [13]		✓	✓		✓
QART	✓	✓		✓	✓

performed; (ii) a set of ontology elements $O_{elements}$, representing relevant classes and properties from $\mathcal{O}$; (iii) a set of few-shot examples $KG_{examples}$ extracted from an existing $\mathcal{KG}$; and (iv) the input summarized text. Formally, we define the prompt as:

$$\mathcal{P} = \{P_{instructions}, O_{elements}, KG_{examples}, text\}$$

Definition 3.7 (Large Language Model (LLM)). A pre-trained neural model designed to generate and understand human-like natural language by leveraging massive text corpora. In our context, an LLM receives a prompt $\mathcal{P}$ and produces a set of RDF triples $\hat{\mathcal{T}}$ as output. Formally, the LLM is a function:

$$LLM : \mathcal{P} \rightarrow \hat{\mathcal{T}}$$

where $\hat{\mathcal{T}}$ is the set of triples generated from the prompt, ideally aligned with the structure defined by the ontology $\mathcal{O}$ and the knowledge graph $\mathcal{KG}$.

3.2 Method

Figure 1 presents our method, which encompasses the following key steps: Extract Ontology Elements, Extract KG Examples, Build Prompt, and Generate Triples. We present our method step by step, and Algorithm 1 formalizes our solution. We assume the basic components of ontologies and KGs as formalized in Section 3.1.

The input of our method is a summarized text. The summarized text represents a unique NL phrase that merges a question asked by an e-commerce customer about a product with an answer provided by a human attendant (e.g., the seller) that answers the question. In particular, this summary is the output of one of the *QART* steps, our framework for generating RDF triples [18]. The output of our method is a set $\mathcal{T}$ of triples that could be added to an existing $\mathcal{KG}$.

At Step 1 (cf. 1 in Figure 1), in addition to the summarized texts, we extract information from an existing ontology $\mathcal{O}$ (line 2 in Algorithm 1). This information is a set of elements from an existing input $\mathcal{O}$ that define statements (classes and properties) in an existing e-commerce $\mathcal{KG}$. We extract key aspects of the ontology, including Cl , Pr , and $\mathcal{T}$ (classes, properties, and triples, respectively). This information is key to Step 3 (cf. 2 in Figure 1) because it defines how the resulting RDF triples should be generated, reducing LLM hallucinations and ensuring that random triples outside the pre-established standards are not created at this stage. The summarized text and ontology elements are the minimum requirements for constructing $\mathcal{P}$ in Step 3.

At Step 2 (cf. 2 in Figure 1), the ontology expert extracts up to n examples from an existing KG. The number of examples is determined by the parameter n , specified by the expert (line 3 of Algorithm 1). These extracted examples serve as few-shot examples in the prompt construction process in Step 3.

Few-shot examples are instances where a language model is given a small set of input-output pairs to guide its response generation. This technique, described in [5], involves presenting the model with a limited number (n) of examples, enabling it to generalize and perform similar tasks on new, unseen data. Few-shot learning (also known as in-context learning) is especially valuable when n is small, as it enables models to learn effectively in scenarios where obtaining large amounts of labeled data is costly or time-consuming.

At Step 3 (cf. 3 in Figure 1), our solution defines a prompt ($\mathcal{P}$) to facilitate NL understanding and RDF triple generation (line 5 in Algorithm 1). The prompt $\mathcal{P}$ is composed of an instruction part ($P_{instructions}$, line 4 of Algorithm 1), the ontology elements ($O_{elements}$), the KG few-shots examples ($KG_{examples}$), and the summarized text (the input).

The rationale for this prompt design is to ensure that the LLM generates RDF triples that are semantically meaningful and ontology-compliant. The instruction part ($P_{instructions}$) guides the task and reduces ambiguity. The few-shot KG examples ($KG_{examples}$) are a reference, helping the model learn the expected pattern and improving its ability to generalize from past cases. The ontology elements ($O_{elements}$) enforce alignment with the KG structure.

After $\mathcal{P}$ is constructed, it serves as input to Step 4, where the defined LLM uses it to generate the RDF triples (line 6 of Algorithm 1).

The process outlined in the image is only responsible for generating RDF triples from summarized text using ontology elements, KG examples, and LLM prompting. However, before these generated triples can be incorporated into an existing KG, they must undergo validation to ensure accuracy, consistency, and relevance. This validation is handled by a separate pipeline consisting of four distinct steps, which assess the semantic correctness of the triples and their compatibility with the existing KG structure. The details of this validation process are beyond the scope of this paper and are discussed in-depth in our separate study [17].

3.3 Running Example

To provide a concrete illustration of our method, we present a running example in which a natural-language text is trans-

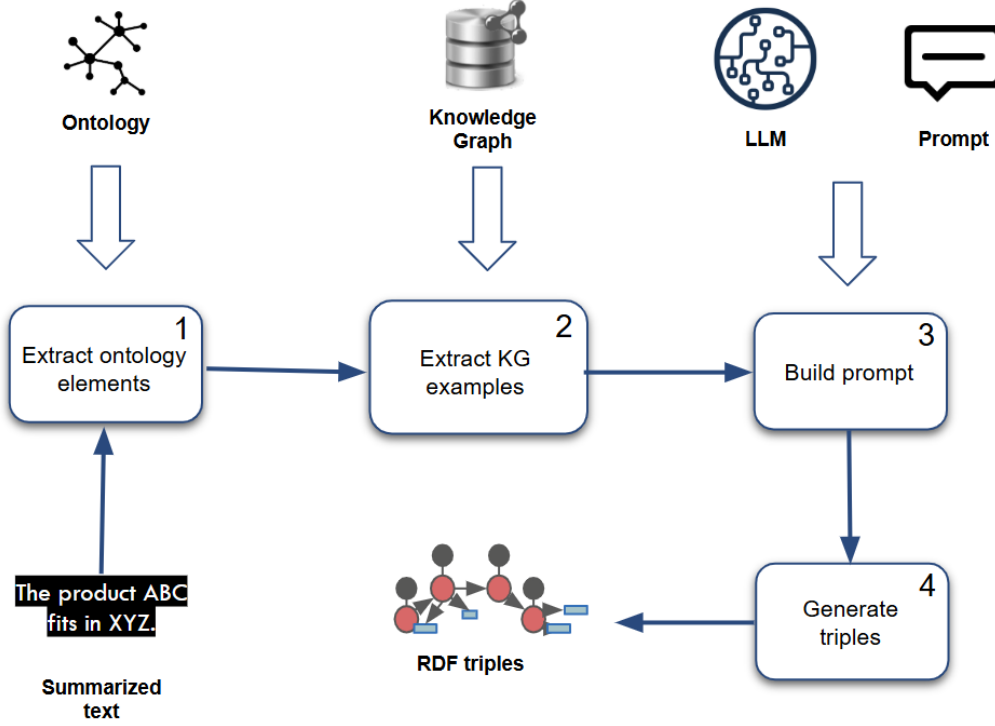


Figure 1. Method to generate RDF triples from e-commerce Q&A texts. The input is a text summarizing the question asked by a customer and answered by a seller in an e-commerce platform. The output is a set $\mathcal{T}$ of triples that describe the summarized text, while respecting statements from $\mathcal{O}$ and an existing KG . The top of this figure presents the inputs to the steps. The boxes represent the steps of our method.

Algorithm 1 RDF Triple Generation

Require: $text, O, KG, n, instructions, LLM$

- | | |
|---|----------|
| 1: procedure GenerateTriples($text, O, KG, n, instructions, LLM$) | |
| 2: $O_{elements} \leftarrow \text{ExtractOntologyElements}(O)$ | ▷ Step 1 |
| 3: $KG_{examples} \leftarrow \text{ExtractKGExamples}(KG, n)$ | ▷ Step 2 |
| 4: $P_{instructions} \leftarrow \text{DefinePromptInstructions}(P, instructions)$ | ▷ Step 3 |
| 5: $P \leftarrow \text{BuildPrompt}(P_{instructions}, O_{elements}, KG_{examples}, text)$ | ▷ Step 3 |
| 6: $generatedTriples \leftarrow \text{LLM_GenerateTriples}(LLM, P)$ | ▷ Step 4 |
| 7: return $generatedTriples$ | |
| 8: end procedure | |

formed into RDF triples. The scenario is as follows: a customer navigates to the product page on a Brazilian e-commerce system that sells a “Car Multimedia System”. The customer, who owns a 2018 Honda Civic, asks about the compatibility of the multimedia system with their car.

QART retrieves a summarized version of a previous answer from the online store. The summarized response, expressed in natural language, simplifies the original question-answer pair into a concise statement indicating whether the product is compatible (“The product with ID 172891 is compatible with the Honda Civic 2018 car”). The description of how the QART framework summarizes the text is not in the scope of this article. We discuss further details about this topic in Regino *et al.* [18].

The method’s first step is to retrieve ontology elements associated with compatibility questions in the automotive domain (Step 1 in Figure 1). An ontology maintainer manually indicated these elements. Classes such as “Car”, “Product”, “Store”, and “Compatibility types” (FullCompatibility, NoCompatibility, ConditionalCompatibility), along with properties like *has:Model*, *has:ModelYear*, *store:sells*, are pre-

registered as essential elements for this domain.

The summarized sentence, the instruction part, the list of relevant ontology elements (Cl and Pr of $\mathcal{O}$), and the few-shot examples serve as input for $\mathcal{P}$ (Step 3 in Figure 1). The prompt includes the following:

1. instructions:

- **context:** “You are an expert in RDF triple generation”;
- **task instruction:** “Create RDF triples from a text that describe the compatibility of a product and a car”;
- **methodology:** “I will provide the classes, properties of the ontology, and a summarized phrase containing a product, a car, and a compatibility type. Use this information to generate the RDF triples”;

2. ontology elements: Classes Cl and Properties Pr ;

3. **few-shots:** “I will provide two examples of phrases and generated triples” followed by two pairs of summarized sentences and their associated T_i ;

4. **summarized sentence:** *The product with ID 172891 is compatible with the Honda Civic 2018 car.*

The LLM generates the following set of triples:

- `<Car/honda-civic-2018> rdf:type onto:Car`
- `<Car/honda-civic-2018> onto:hasBrand "honda"`
- `<Car/honda-civic-2018> onto:hasModel "civic"`
- `<Car/honda-civic-2018> onto:hasModelYear "2018"^^xsd:integer`
- `<Product/store_name/id-158> rdf:type onto:Product`
- `<Product/store_name/id-158> onto:hasMLID "ID158"`
- `<Product/store_name/id-158> onto:hasFeature "Bluetooth"`
- `<Store/store_name> onto:sells <Product/store_name/id-158>`
- `<FullCompatibility/store_name/id-158/honda-civic-2018> rdf:type onto:FullCompatibility`
- `<Product/store_name/id-158> onto:hasCompatibility <FullCompatibility/store_name/id-158/honda-civic-2018>`
- `<FullCompatibility/store_name/id-158/honda-civic-2018> onto:compatibleWith <Car/honda-civic-2018>`

The new triples are ready to be syntactically and semantically validated in the next step of QART and, if no errors are found, can be inserted into the KG.

4 Evaluating Ontology-Defined Triples Task

This section describes an experimental evaluation to assess our solution in the e-commerce scenario. Section 4.1 contextualizes our experimental evaluation. Section 4.2 presents all necessary materials for our experiments, including the datasets and models. Section 4.3 presents the designed evaluation methodology to obtain the results. Section 4.4 reports on the quantitative analysis of results, whereas Section 4.5 is dedicated to demonstrating a qualitative study.

4.1 Context

This experimental evaluation aimed to assess the effectiveness of our RDF triple-generation proposal within the specific context of e-commerce. We conducted our experiments on the most important step of our solution: generating triples using language models. In this sense, the tasks related to checking for redundant and semantically inconsistent triples were not included in this evaluation.

The company responsible for managing the datasets used in the experiments described in this section, called GoBots, aims to build artificial intelligence solutions for e-commerce. Its main customers are automotive-related stores. The KG created and maintained by the company is designed to handle compatibility questions and answers in this industry [21].

On this basis, we understand that the company has considerable variability in its automobile compatibility data, which

facilitates the creation of gold-standard datasets to evaluate experiments. By conducting eight distinct experiments, we seek to understand the LLM's ability to consistently generate triples across different stores and to assess its generalizability by varying aspects of the dataset, prompt, and model. Figure 2 presents the materials and methods (distinct setups) used to generate the experimental results.

4.2 Materials

The materials include a list of datasets $\mathcal{D}$ and gold standard datasets $\mathcal{GS}$, an ontology $\mathcal{O}$, two large language models LM and two prompts $\mathcal{P}$ (cf. Figure 2).

Our evaluation datasets ($\mathcal{D}$) were constructed from information extracted from GoBots' e-commerce customers³. Each dataset has a different number of rows, but all have the same columns. The columns are question, answer, and generated triples. Figure 3 presents an example of a row from one of the datasets. The question column (first line of Figure 3) indicates the question asked by the customer, and the answer field (second line of Figure 3) suggests the answer given by the attendant to the question.

The generated triples field (third line of Figure 3) shows the triples related to the question and answer. To improve data visualization and understanding, we organized the nine generated triples into four groups (cf. Figure 3): in green, triples related to the store; in pink, triples related to the product being sold by the seller; in blue, triples related to the customer's car and in yellow, triples related to compatibility between the product and the car.

Creating these triples required additional text processing (e.g., intent and entity identification), as further explained in Sant'anna *et al.* [21]. These triples were annotated and evaluated manually [21]. We used the generated triples column in the gold standard dataset ($\mathcal{GS}$) to assess the quality of the triples generated by our solution. This set of triples is in a production environment and has been validated by the $\mathcal{KG}$ maintainers. This gold standard ensures consistency across our evaluation processes, thereby promoting reliable, comparable results.

The first dataset (D_1) comprises 500 rows of compatibility questions from a single automobile store (cf. Figure 2). This dataset aimed to provide an in-depth assessment of our solution's effectiveness within a specific context and store. The second dataset (D_2) includes 420 compatibility questions from ten different automobile stores, offering insights into the solution's generalizability across a diverse range of e-commerce stores (cf. Figure 2). The stores and questions were randomly selected, yielding 42 questions per store. The questions from D_1 and D_2 were created in 2023.

The ontology (O_1) serves as the foundational knowledge structure that drives the generation of RDF triples. It comprises essential classes (C) and properties (Pr), such as Store, Product, Car, and Compatibility Answer (cf. Figure 2). The R&D team of GoBots created this ontology and is currently used to represent the knowledge of the GoBots' customer compatibility questions and answers [21]. Table 2 presents the list of the most relevant classes in O_1 .

³At no time were customer data or privacy compromised through access

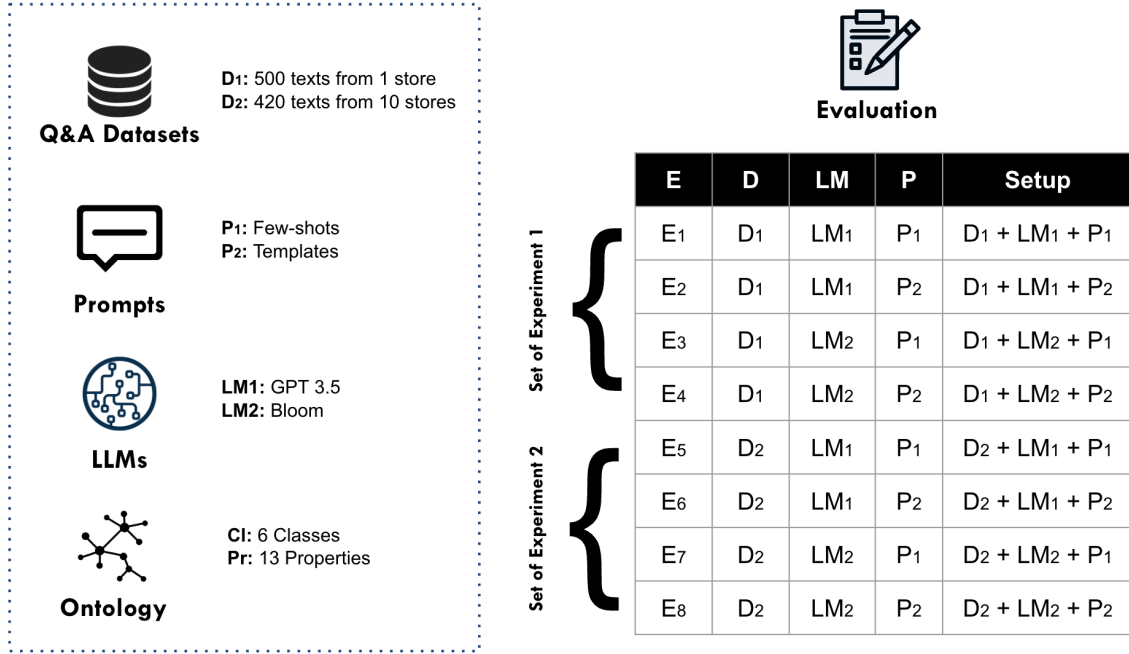


Figure 2. Experimental evaluation to verify our method capability to generate RDF triples from different input variations. In the left part of the figure, we have the materials: two datasets, two prompts, two LLMs, and an ontology. On the right, a matrix that combines the materials and generates 8 test setups. The first column identifies the test setups (from E_1 to E_8). The last column of this matrix describes each of these setups (e.g., the second line shows that the first dataset was used, along with the first LLM in conjunction with the second prompt, resulting in $D_1 + LM_1 + P_2$).

Q: Does it fit in HRV 2021?

A: Yes, perfectly. We are awaiting your purchase.

T: <Store/store_name> onto:sells <Product/store_name/ml-id-mlb1585022681> .

<Product/store_name/ml-id-mlb1585022681> rdf:type onto:Product .

<Product/store_name/ml-id-mlb1585022681> onto:hasMLID "MLB1585022681" .

<Car/hrv-2021> rdf:type onto:Car .

<Car/hrv-2021> onto:hasModel "hrv" .

<Car/hrv-2021> onto:hasModelYear "2021"^^xsd:integer .

<FullCompatibility/store_name/ml-id-mlb1585022681/hrv-2021> rdf:type onto:FullCompatibility .

<Product/store_name/ml-id-mlb1585022681> onto:hasCompatibility <FullCompatibility/store_name/ml-id-mlb1585022681/hrv-2021> .

<FullCompatibility/store_name/ml-id-mlb1585022681/hrv-2021> onto:compatibleWith <Car/hrv-2021> .

Figure 3. Example of a row in one of our evaluation datasets. The letter Q is followed by the question asked by a customer; the letter A is followed by the answer provided by the human attendant; the letter T is followed by nine triples related to Q and A . The triples use four main classes (Cl) of our evaluation ontology: Store (green), Product (pink), Car (blue), and Compatibility (yellow).

Table 2. Classes and properties of O used in the experimental evaluation.

	Class	Property
Store	Store	onto:sells
		onto:Car
		has:model
Car	Car	has:modelYear
		has:fabrificationYear
		has:Brand
Product	Product	onto:Product
		has:MLID
		onto:compatibleWith
	FullCompatibility	has:Compatibility
Compatibility	ConditionalCompatibility	onto:FullCompatibility
	NoCompatibility	onto:ConditionalCompatibility
		onto:NoCompatibility

In our case, we assume that the LLM possesses the contextual knowledge to comprehend compatibility questions and to this dataset.

answers, translating them into structured triples that capture the relationships between “products” and “cars”. We used GPT-3.5 from OpenAI as the first Large Language Model

(LM_1) and Bloom [22] as the second (LM_2) (cf. Figure 2). These LLMs were chosen based on state-of-art results reported on text generation tasks [5] [22]. They were trained with 176 and 175 billion parameters, respectively. Both models are accessible via API and do not require the loading and storage of the model on our server, which facilitates the task of using LLMs by avoiding the configuration and provisioning of our infrastructure.

We set 0.1 as the hyperparameter for both LM_1 and LM_2 . The temperature is a hyperparameter ranging from 0 to 1, where values close to 1 give the LLM greater freedom to generate text beyond the scope defined by the prompt. In contrast, values close to 0 restrict this freedom, making the prompt more deterministic and potentially reducing hallucinations. In our context, the more the LLM adheres deterministically to the directions specified in the prompt, the more triples are generated that adhere to ontological classes and properties. For this reason, we chose a low temperature value.

There is a gold standard ($\mathcal{GS}$) for each dataset. Summarized sentences form them, and triples are created a priori. These triples have been validated by GoBots customers and are being used in production to generate automobile-compatibility responses. $\mathcal{GS}$ is used to evaluate the extent to which our method is accurate regarding generated properties, classes, and triples.

The last material used in all the experimental evaluations is the prompts (cf. Figure 2). It provides context, instructions, and examples for the LLM to follow. Figure 4 presents the prompts used in our experiments: P_1 , identified by the title “Few Shots”, and P_2 , determined by the title “Templates”.

The P_1 prompt is formed by an instruction part (numbers 1, 2, and 3 inside the “Prompt 1 - Few Shots” section of Figure 4) and a few-shot example part (number 4 at the top of Figure 4). We organized the instruction part into three sections to improve readability.

1. The first section (number 1 of Figure 4) describes the context, the role of the LLM - RDF triple generator - and the main task it should address - generate RDF triples from the text.
2. The second section (number 2 of Figure 4) describes the input composition.
3. The third section provides the classes and properties that should be presented in the final generated triples. Due to space limitations, we replace the list of classes and properties in Figure 4 with placeholders `<list_of_classes>` and `<list_of_properties>`. In the original prompt, we listed all the classes and properties in Table 2.
4. The fourth section (few-shot section – number 4 in Figure 4) receives two examples of sentences and triples associated with it. We adopted the use of placeholders in Figure 4 - similar to the classes and properties definition section. The summarized text and triples are condensed in the `<summarized_sentence>` and `<list_of_triples>`, respectively. The list of triples used as a few shots example is similar to the triples described in Figure 3. The rationale for using a few shots is that they improve results compared to using only instructions [5]. Adding examples helps LLM generate outputs closer to the ex-

pected standard.

The second prompt (bottom of Figure 4) describes how we used templates instead of a few-shot learning technique (explored in P_1) to generate RDF triples. It is organized in two parts, identified by numbers 1 and 2.

1. The first part explains the task (generating triples) and the methodology (using templates), based on an input (summarized texts and triples in template format).
2. The second part defines the list of triples with many placeholders. The LLM reads the summarized sentence, extracts relevant parts of it (e.g., car model, type of compatibility), and replaces the templates with it (`{CAR_MODEL}` and `{TYPE_OF_COMPATIBILITY}` should be replaced by some values present in the summarized sentence, like “Corolla” and “FullCompatibility”).

The rationale for investigating the use of the prompt P_2 is to check whether templates and the absence of a list of classes, properties, and examples (as explored in the prompt P_1) produce comparable or better results than the approach encoded in the prompt P_1 . If this hypothesis is proven, only templates may be sufficient to generate triples from the summarized sentences.

4.3 Evaluation Methodology

Figure 2 presents an overview of the experimental evaluation process. We used D_1 and D_2 for experiment 1 and experiment 2, respectively.

The table (evaluation) in Figure 2 shows all our combinations among the datasets, prompts, and LLMs: $D \times LM \times P$. This yielded eight sets of results (experimental setups), 4 for D_1 and 4 for D_2 . For each of the eight sets, we added an identifier E_n , n ranging from 1 to 8 (cf. first column of the table in Figure 2).

In the first experiment, we targeted the dataset D_1 , containing a specific store within the automobile domain (E_1, E_2, E_3, E_4 of Figure 2). We started by transforming the question-and-answer columns of D_1 into a single summarized phrase indicating whether the products are compatible. Further details of this choice can be found in our recent work [18].

With the summarized text ($\mathcal{ST}$) from each Q&A pair, we proceed to the prompt $\mathcal{P}$ construction. In this experiment, we use both P_1 and P_2 . To construct P_1 (cf. Figure 4), we created the instruction part containing the list of essential classes and properties (cf. Table 2). We selected two random examples in the few-shot example part of P_1 . We append each $\mathcal{ST}$ at the end of P_1 , using $\mathcal{LM}$ to generate the output: the list of generated triples by each $\mathcal{ST}$. For P_2 , we did not have to provide the list of classes, properties, and examples as in P_1 . Instead, we used the templates described in the second part of the description of Prompt two P_2 presented in Figure 4.

We decided to evaluate the use of P_1 and P_2 in two different LLMs: GPT (LM_1) and Bloom (LM_2). This approach ensures a robust evaluation by considering the performance across different language models, offering a more nuanced

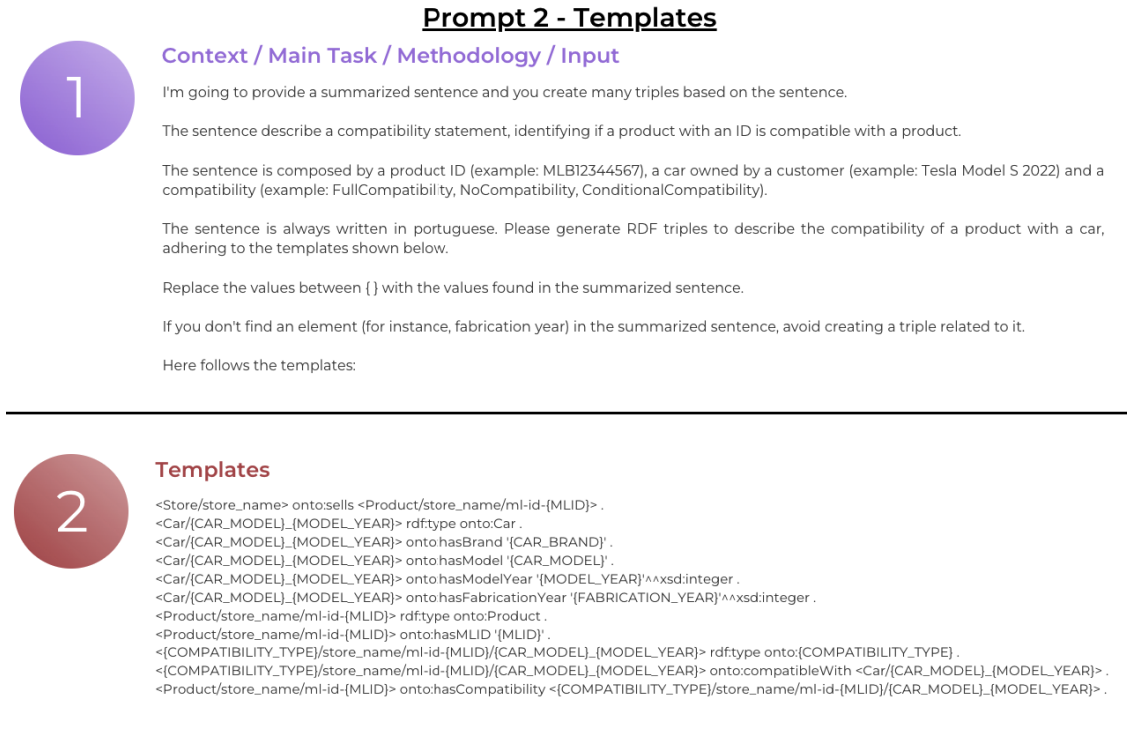
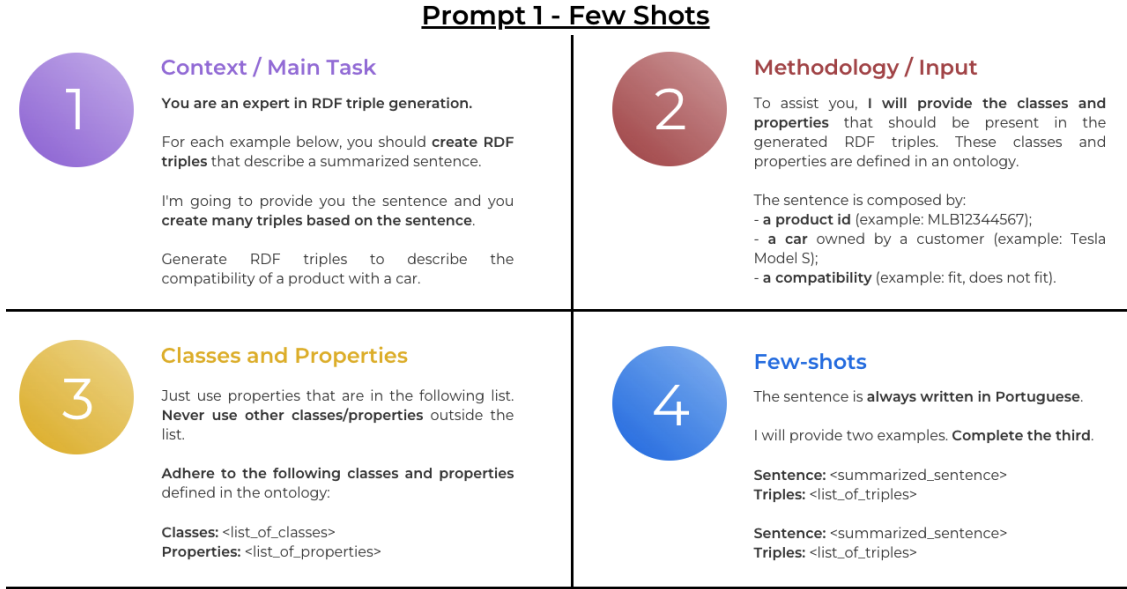


Figure 4. Prompts used in the experimental evaluation. Prompt 1, at the top of this figure, is composed of four steps. Prompt 2, in the bottom this figure, is composed of two steps.

understanding of the methodology's effectiveness. Assessing diverse LLMs helps uncover potential model-specific variations and strengths, contributing to the reliability and generalizability of the results.

The methodology applied in experiment 2 replicated the procedures from the first experiment, ensuring consistency in evaluation criteria and the overall approach. We used LM_1 , LM_2 , P_1 and P_2 for this experiment. The difference with the first experiment is that we used the D_2 instead of D_1 (E_5 , E_6 , E_7 , E_8 of Figure 2).

To evaluate the generated triples, we compared them with the $\mathcal{GS}$ triples. We measure the overlap of classes, properties

(from Table 2), and triples. The rationale is that if our method can generate atomic parts (classes and properties) and triple parts (RDF triples) that are similar to our gold standard, we can consider it a good candidate for creating triples for specific stores and domains.

In summary, we aimed to identify the following aspects:

- Whether all the classes listed in Table 2 were used in the resulting triples;
- Whether all the properties listed in Table 2 were used in the resulting triples;
- Whether there are the resulting triples in the correct

triple syntax;

- Whether there are exceeding triples that should not have been created, with properties and classes not present in Table 2.

4.4 Quantitative Analysis Results

This subsection presents the quantitative analysis of the results. Table 3 presents the accuracy results of the set of experiment 1 (with D_1 , including E_1, E_2, E_3, E_4) and Table 4 presents the accuracy results for the set of experiment 2 (with D_2 , including E_5, E_6, E_7, E_8). Both are organized into three sections: "Classes", "Properties", and "Triples". Each section contains specific entities with corresponding values under different prompts (P_1 and P_2) for GPT and Bloom (LM_1 and LM_2), along with the gold standard (GS) values. GS values serve as ground truth, a reference point for evaluating the models' effectiveness in generating triples.

Under the "Classes" group in Table 3 and Table 4, we counted the resources for the classes listed in Table 2: Store, Car, Product, and three types of compatibility. The same occurs for the "Property" group. For the "Triples" group, we aimed to identify if the whole triples - composed of the classes and properties - were correctly constructed.

We use accuracy to measure how many elements in the gold-standard triples are also present in the triples generated by our framework. We consider accuracy as a metric for identifying exact matches. The closer these numbers are, the more similar our method's results are to the expected ones. We can identify cases where more elements were generated than necessary and also cases where fewer elements were generated than necessary. Both cases warrant attention for the incompleteness of element generation and for the model's hallucination, creating elements that should not appear in the resulting triples.

We identified in Experiment One that the most accurate prompt for D_1 is P_1 , the prompt that uses the few-shot technique as highlighted in bold in Table 3, P_1 reached better accuracy in 6 out of 6 classes (100%), 11 out of 13 properties (84.61%), and 9 out of 9 triples (100%). Examining the properties in which P_1 had a worse accuracy, we identified *onto:car* and *onto:hasFabricationYear*. For *onto:car*, we considered the accuracy slightly worse than the best accuracy of P_2 (497 out of 500 and 498 out of 500, respectively). For *onto:hasFabricationYear*, there is a clear tendency to identify the vehicle's year of manufacture when there is no evidence of this property in the GS. This behavior was identified in all LM and P for the *onto:hasFabricationYear* property in D_1 , which was accentuated in P_1 . We elucidate this case in more detail in Section 4.5.

The model that, together with P_1 , achieved the highest accuracy was LM_2 , Bloom (cf. Table 3). LM_2 achieved higher accuracy in 3 out of 6 classes (50%), 8 out of 13 properties (61.53%) and 8 out of 9 triples (88.89%). Examining in more detail, we identified that LM_2 had difficulties in identifying classes and properties related to compatibility (*FullCompatibility*, *NoCompatibility*, and *ConditionalCompatibility*). Many cases of *FullCompatibility* and *onto:fullCompatibility* have been classified as *ConditionalCompatibility* and *onto:conditionalCompatibility*, respec-

tively. Section 4.5 further elucidates this case.

In the set of experiment 2 (with the dataset D_2), we identified results similar to the set of experiment 1, with the highest accuracies achieved by P_1 (cf. Table 4). P_1 achieved better accuracy in 5 out of 6 classes (83.33%), 7 out of 13 properties (53.84%) and 9 out of 9 triples (100%). All properties of P_1 that did not achieve the highest accuracy revealed results close to the highest accuracy, which we understand as an adequate result.

The model that, together with P_1 , achieved the highest accuracy was LM_2 , Bloom (cf. 4). LM_2 achieved higher accuracy in 3 out of 6 classes (50%), 5 out of 13 properties with P_1 and P_2 (38.46%) and 8 out of 9 triples (88.89%). As in set of experiment 1, we identified that LM_2 with D_2 did not produce good results with classes and properties related to the type of compatibility.

Concerning P_2 , in both Experiments 1 and 2, we identified a problem with the templates as an approach to the prompt. P_2 , as described in Figure 4, uses a brief description, followed by templates, instead of a few shots as in P_1 . The rationale is that the responses of both LM_1 and LM_2 contained the values associated with the templates (e.g. 2018, Honda, ID2018719, FullCompatibility) instead of the templates (e.g. MODEL_YEAR, CAR_MODEL, MLID). Everything between should be replaced. This behavior of replacing the template with the associated value occurred in the OpenAI model (LM_1). In many triples resulting from LM_2 , the templates were not replaced.

4.5 Qualitative Analysis Results

This subsection presents a qualitative study. To clarify the effectiveness of our solution, we present single examples of e-commerce texts from our datasets and the automatic transformation into RDF triples using our developed technique.

We selected specific examples that illustrate key aspects of the model's behavior. The examples were not chosen randomly but were instead deliberately selected based on two main criteria:

- Representativeness of Identified Patterns – We looked for examples that exemplify common or recurring patterns observed in our experiments, particularly those that highlight systematic issues such as hallucinations, misclassifications, or deviations from the expected output;
- Illustration of Unexpected or Critical Errors – We prioritized cases where the generated triples exhibited particularly interesting or unexpected behaviors. These cases provide deeper insight into the model's reasoning process, highlighting potential pitfalls that may not be evident from quantitative metrics alone.

By focusing on these criteria, we aimed to provide a richer understanding of the model's strengths and weaknesses, rather than relying on arbitrary selections.

Example 1: Fabrication Year Hallucination

As described in Subsection 4.4, there were several cases of the creation of the triple referring to the year of manufacture of the customer's car, even if there was no such information

Table 3. Results for Experimental Evaluation One comparing GPT and Bloom for triple generation task. We evaluated D_1 , a dataset with 500 rows of a single automobile store. The first column shows all the entities evaluated, divided in three groups: classes, properties and triples; the second and third columns describe the number of instances found using GPT and Bloom with P_1 ; the fourth and fifth columns describe the number of instances found using GPT and Bloom with P_2 , the last column shows the number of instances in the Gold Standard dataset. In bold, the best accuracy result for each class, property and triple.

Entities	Prompt 1		Prompt 2		GS
	GPT	Bloom	GPT	Bloom	
Classes					
Store	494	500	309	415	500
Car	493	497	473	423	500
Product	494	500	473	425	500
FullCompatibility	300	105	262	403	281
NoCompatibility	53	75	107	80	44
ConditionalCompatibility	151	301	128	27	175
Properties					
onto:car	493	497	473	498	500
onto:product	494	500	473	498	500
onto:hasModel	492	500	473	496	500
onto:hasModelYear	491	497	472	496	500
onto:hasFabricationYear	237	234	154	496	39
onto:hasBrand	328	253	473	496	91
onto:hasMLID	494	500	473	497	500
onto:compatibleWith	494	495	373	472	500
onto:hasCompatibility	494	500	473	450	500
onto:sells	494	500	309	419	500
onto:fullCompatibility	301	96	250	358	281
onto:noCompatibility	53	72	108	78	44
onto:conditionalCompatibility	152	286	129	24	175
Triples					
(Car rdf:type Car)	493	497	473	289	500
(Product rdf:type Product)	494	500	473	55	500
(Product hasCompatibility Compatibility)	494	500	473	48	500
(Car compatibleWith Compatibility)	493	491	373	46	500
(Car hasModel model)	491	497	473	294	500
(Car hasModelYear year)	491	497	472	247	500
(Car hasBrand brand)	327	253	473	295	91
(Store sells Product)	494	500	309	52	500
(Product hasMLID MLID)	494	500	473	70	500

in the summarized sentence used as input for the models. According to Table 3, 237 triples were found with the property *onto:hasFabricationYear* using the combination $LM_1 + P_1$, 234 for the combination $LM_2 + P_1$, 154 for the combination $LM_1 + P_2$ and 496 for the combination $LM_2 + P_2$. The correct, according to the Gold Standard dataset, would be 39 triples. Similar behavior occurred with D_2 .

Examining one of these cases, we understand and hypothesize the reason for this error. We consider this case as a type of hallucination from LM_1 and LM_2 , as it created information that did not exist, instead of the model responding that it did not know the manufacturing year, for example.

The case we examined has the following summarized sentence, translated from Portuguese: “*MLB2154017658 fits on the front doors of Celta 2012 1.0.*”. All triples related to the Car class were correctly created, exactly like the GS. They are:

- `<Car/celta-2012> rdf:type onto:Car .`
- `<Car/celta-2012> onto:hasModel "celta" .`

- `<Car/celta-2012> onto:hasModelYear "2012"^^xsd:integer .`

However, the triple statement that the manufacturing year is also 2012 was created. This is false, as the year in the summary sentence refers to the model as a social convention.

An example of correct detection of *modelYear* and *fabricationYear* is in the summarized phrase: “*MLB2198690553 fits on Fox 2010/2011.*”. This correctly generated the RDF triples:

- `<Car/fox-2010-2011> onto:hasModelYear "2011"^^xsd:integer .`
- `<Car/fox-2010-2011> onto:hasFabricationYear "2010"^^xsd:integer .`

We did not find a pattern in the answers to indicate when this happened. For this reason, we considered it essential to include several mechanisms to verify the triples generated by our solution.

Table 4. Results for Experimental Evaluation Two comparing GPT and Bloom for triple generation task in the automobile domain. We evaluated D_2 , a dataset with 420 rows of ten automobile store. The first column shows all the entities evaluated, divided in three groups: classes, properties and triples; the second and third columns describe the number of instances found using GPT and Bloom with P_1 ; the fourth and fifth columns describe the number of instances found using GPT and Bloom with P_2 , the last column shows the number of instances in the Gold Standard dataset. In bold, the best accuracy result for each class, property and triple.

Entities	Prompt 1		Prompt 2		GS
	GPT	Bloom	GPT	Bloom	
Classes					
Store	411	417	250	356	420
Car	411	416	395	364	420
Product	411	419	395	365	420
FullCompatibility	274	105	235	311	231
NoCompatibility	69	87	111	93	47
ConditionalCompatibility	82	212	70	26	142
Properties					
onto:car	411	413	395	416	420
onto:product	411	417	395	418	420
onto:hasModel	409	413	394	420	420
onto:hasModelYear	409	413	394	420	420
onto:hasFabricationYear	284	190	203	420	54
onto:hasBrand	310	194	394	420	92
onto:hasMLID	411	416	395	419	420
onto:compatibleWith	411	412	295	372	420
onto:hasCompatibility	411	419	395	361	420
onto:sells	411	417	250	361	420
onto:fullCompatibility	274	97	226	268	231
onto:noCompatibility	69	83	111	88	47
onto:conditionalCompatibility	83	200	70	24	142
Triples					
(Car rdf:type Car)	411	413	395	230	420
(Product rdf:type Product)	411	417	395	61	420
(Product hasCompatibility Compatibility)	411	419	395	42	420
(Car compatibleWith Compatibility)	411	410	295	40	420
(Car hasModel model)	409	413	394	235	420
(Car hasModelYear year)	408	413	391	199	420
(Car hasBrand brand)	309	194	394	235	92
(Store sells Product)	411	417	250	53	420
(Product hasMLID MLID)	411	416	395	74	420

Example 2: Full Compatibility vs. Conditional Compatibility

As described in Subsection 4.4, there were several cases where the $\mathcal{LM}$ confused compatibilities, mainly *FullCompatibility* with *NoCompatibility*. This happened independently of the $\mathcal{LM}$, $\mathcal{P}$, and $\mathcal{D}$ used, that is, for all $E = E_1, E_2, \dots, E_8$. A first example of this case was found in E_5 . The summarized sentence, the input for our method, says: “The MLB1860182861 is compatible with the Honda Fit 2017/2018”. We have a clear example of a sentence related to the *FullCompatibility* class and the *onto:fullCompatibility* property. Therefore, one of the triples created for this sentence in the Gold Standard dataset was:

- $\langle \text{Product/store-name/id-18601} \text{ } \text{onto:hasCompatibility} \text{ } \langle \text{FullCompatibility/store-name/id-18601/fit-2017-2018} \rangle .$

We observed in the setup T_5 for this summarized sentence the creation of the following triple:

- $\langle \text{Product/store-name/id-18601} \rangle \text{ } \text{onto:hasCompatibility} \text{ } \langle \text{ConditionalCompatibility/store-name/id-18601/fit-2017-2018} \rangle .$

In other words, the created triple should be related to *FullCompatibility*, but it was identified as *ConditionalCompatibility*.

The second example within the theme of confusion between *FullCompatibility* and *ConditionalCompatibility* deals with the opposite of the case mentioned above: the generated triple should contain the *ConditionalCompatibility* class but contains the *FullCompatibility* class. Here is the summarized sentence, the correct triple according to the Gold Standard, and the triple generated by our method:

- “The ID 185 fits the 2015 tracker with 2 buttons”;
- $\langle \text{Product/store-name/id-185} \rangle \text{ } \text{onto:hasCompatibility} \text{ } \langle \text{ConditionalCompatibility/store-name/id-185/tracker-2015} \rangle .$

- `<Product/store-name/id-185> onto:hasCompatibility`
`<FullCompatibility/store-name/id-185/tracker-2015> .`

We derived some analyses based on these examples and examined the resulting triples:

- The first hypothesis, for the first example (Example 1: Fabrication Year Hallucination), is that the $\mathcal{LM}$ hallucinated when encountering the expression 2017/2018, perhaps interpreting erroneously that the product would serve for Honda Fit cars from the years 2017 or 2018, but for other years, there could be other conditions.
- For the second example (Example 2: Full Compatibility vs. Conditional Compatibility), the Gold Standard understands the expression "2 buttons" in the summarized sentence as a conditional. Thus, the *FullCompatibility* class would not be appropriate, but rather *Conditional-Compatibility*. We hypothesize that the $\mathcal{LM}$ failed to understand this part of the sentence as a conditional.

Example 3: Template Replacement

We assessed P_2 (the template prompt) in 4 setups: E_2 , E_4 , E_6 , and E_8 (Figure 2). In E_2 and E_6 , we used LM_1 , the GPT. We found results close to the $\mathcal{GS}$. Therefore, we considered the use of templates with GPT satisfactory. However, we obtained worse results when combining P_2 with LM_2 (both in E_4 and E_8) than in other E_n , especially in the generated triples assessment.

In Table 4, we observe in the penultimate column (Bloom + Prompt 2) that the E_8 combination correctly identified 230 out of 420 triples in the pattern *Car rdf:type Car* and 61 out of 420 triples in the pattern *Product rdf:type Product*.

Examples of generated triples for a car and a product:

1. `<HondaNewCivic2008> rdf:type onto:Car .`
2. `<ID1834939237> onto:hasID 'ID1834939237' .`
3. `<Car/{CAR_MODEL}_{CAR_YEAR}> rdf:type`
`onto:Car .`
4. `<Product/store_name/id-{ID}> rdf:type onto:Product .`

In the first two examples, the template was not followed. We defined through the template that the class should be in the URI (e.g the class *Store* should appear in the triples with subject, product or object containing store information, like `<Store/store_name/> onto:sells` `<Product/store_name/id-1834939237>`). In the third and fourth examples, there was no change to the template values for the car model, year, or product ID. The correct triples, according to the Gold Standard, for these examples should be:

1. `<Car/civic_2008> rdf:type onto:Car .`
2. `<Product/id-1834939237> onto:hasID 'ID1834939237'`
`.`
3. `<Car/hrv_2023> rdf:type onto:Car .`
4. `<Product/store_name/id-1010291> rdf:type`
`onto:Product .`

We understand that incorrect template substitution (examples 1 and 2) and the lack of template substitution (examples 3 and 4) can contaminate the KG if added without verification. Examples where no template substitution invalidates

the triple and the semantic role within the KG, as they would not add any knowledge about the text. One way to avoid such behaviors is to ensure that the template (everything between the `{ }` characters) has been replaced and that there are no triples without the required classes.

5 Evaluating Multidomain Triple Generation Task

This section presents the experimental evaluation of our solution in an e-commerce multidomain scenario. Section 5.1 contextualizes the experimental assessment. Section 5.2 presents the datasets. Section 5.3 presents the designed evaluation methodology to obtain the results. Section 5.4 reports on the quantitative analysis of results.

5.1 Context

By conducting 32 experimental settings, we seek to understand the influence of e-commerce domains, intents, LLMs, and prompt type in the triple generation task. Figure 5 presents the materials and methods (distinct setups) used to generate the experimental results.

5.2 Materials

This subsection describes the materials used in our evaluation, including datasets, prompts, language models, intents, and the gold standard used for comparison.

Datasets (D_1 and D_2): The evaluation was conducted using two datasets. The first dataset (D_1) consists of 300 summarized texts within a single domain: automobiles. Each text represents a factual summary of an NL question a customer asked and a seller's corresponding response. The second dataset (D_2) includes 300 summarized texts spanning 20 domains (e.g clothing, automobiles, electronics, and furniture). Like D_1 , each summarized text consolidates the product name, question, and answer into a single sentence generated in the first step of our framework, QART.

Both datasets were created by randomly selecting rows from GoBots' dataset, covering customer interactions between January and December 2024. The texts and generated triples are written in Brazilian Portuguese. The rationale for creating these datasets is to investigate whether the LLM can effectively handle multiple domains and assess the impact of summarizing sentences from diverse domains on triple-generation quality.

Prompts (P_1 and P_2): Two types of prompts were employed for triple generation. The first (P_1 , zero-shot prompt) consists of a contextual introduction, an initial instruction, and a set of constraints guiding the language model in generating RDF triples without any example references. The second (P_2 , few-shot prompt) includes, in addition to the components of P_1 , three example cases to illustrate the expected structure and style of the generated triples, helping the language model produce more accurate results. As a rationale, we seek to determine whether including examples in the few-shot prompt helps the LLM generate triples that more closely align with the gold standard.

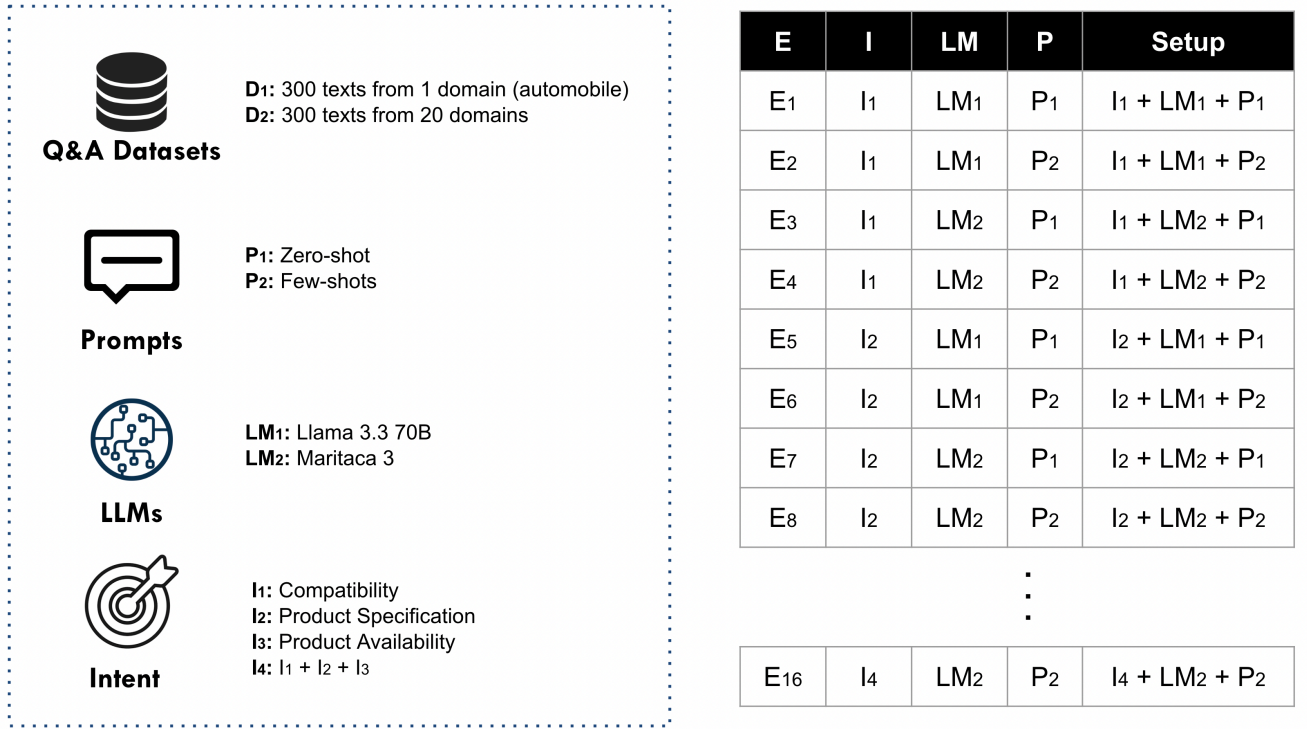


Figure 5. Experimental evaluation to check multidomain triple generation. In the left part of the figure, we have the materials: two datasets, two prompts, two LLMs and four intents. On the right, a matrix combines the materials and generates 16 test setups for the first dataset (D_1). The first column identifies the tests setups (from E_1 to E_{16}). The last column of this matrix describes each of these setups (e.g. the second line shows that the first intent was used, along with the first LLM in conjunction with the second prompt, resulting in $I_1 + LM_1 + P_2$). The second dataset (D_2) has the same matrix of evaluation setup, with the difference that it uses D_2 instead of D_1 . The experiments using D_1 and D_2 results in 32 experimental settings (from E_1 to E_{32}).

Language Models (LM_1 and LM_2): The triple generation process used two large language models. The first, LM_1 , represents LLaMA 3.3 (70B parameters) from Meta, a state-of-the-art open-source multilingual model with robust factual consistency. The second, LM_2 represents Sabia-3, a Brazilian Portuguese fine-tuned LLM specifically optimized for Portuguese-language tasks [1].

We employed a conservative temperature setting of 0.1 for both models to ensure deterministic and controlled outputs. Our rationale for choosing these LLMs was to compare a widely used multilingual model with a Portuguese fine-tuned model to determine whether a language-specific model performs better than a general multilingual one.

Intents (I_1 to I_4): The e-commerce queries in the dataset were categorized into four intent groups. The compatibility intent (I_1) is found in customer inquiries about product compatibility. The specification intent (I_2) is reflected in questions about product specifications, such as size, voltage, and weight. The availability intent (I_3) is found in inquiries regarding stock availability, color variations, and other availability-related aspects. The mixed intents (I_4) dataset contains a mix of the three previous intent types. Each intent category (from I_1 to I_3) includes 100 sentences in both D_1 and D_2 , totalling 300 summarized sentences, 100 for each intent category.

As a rationale for intent categorization, we aimed to examine whether LLMs handle different types of entities with varying ease and whether mixed-intent datasets introduce confusion during triple generation.

Gold Standard (GS_1 and GS_2): The gold standard

dataset was created using both single-domain (D_1) and multi-domain (D_2) datasets. The process involved three annotators – an undergraduate, a PhD student, and a recently graduated PhD – each with prior experience in semantic web and triple construction. Each annotator was given a spreadsheet containing the summarized sentences and one month to generate one RDF triple per sentence that best captured its core meaning. These manually constructed triples serve as a reference to evaluate the quality of the triples generated by the LLMs. We assess the alignment between automatically extracted knowledge and expert-validated data by comparing LLM-generated triples with the human-created gold-standard triples.

5.3 Evaluation Methodology

Figure 5 presents an overview of the experimental evaluation process. We used D_1 and D_2 for experiments 1 and 2, respectively.

Table (evaluation) in Figure 5 shows all our combinations among the intents, prompts, and LLMs for experiment 1 (with D_1): $I \times LM \times P$. This generated sixteen sets of results (experimental setups). For experiment 2, we have an additional sixteen sets of results. For each of the third two sets, we added an identifier E_n , n ranging from 1 to 32 (cf. first column of the table in Figure 2).

In the first experiment, we targeted the dataset D_1 , which contains many stores in the automobile domain. With the summarized text (ST), we proceed to the prompt P construction. In this experiment, we use both P_1 and P_2 . To construct

P_1 , we created the initial directions and instruction part. For P_2 , we selected three random examples in the few-shots example part of P_2 . We append each ST at the end of P_1 and P_2 , using $\mathcal{LM}$ to generate the output: the list of generated triples by each ST .

In summary, we aimed to identify the following aspects:

- Whether the number of domains (one or many) processed by the LLM impacts the resulting triples and how;
- Whether the number of examples provided in the prompt (zero or three) impacts the resulting triples and how;
- Whether the language specificity of the LLM (multilingual or Portuguese fine-tuned) impacts the resulting triples and how;
- Whether the variability of intents (one intent or three) impacts the resulting triples and how.

We employed two metrics to assess the similarity between the generated triples and the gold standard: *BLEU* [16] and *BERTScore* [30]. The BLEU Score (Bilingual Evaluation Understudy) is a precision-based metric that measures n-gram overlap between the generated triples and the gold standard. It compares unigrams, bigrams, trigrams, and fourgrams of the generated and reference triples. It ranges from 0 to 1, where 1 means a perfect word-for-word match. We use it to evaluate the surface-level similarity of triples, providing a complementary perspective to the semantic metrics.

Using contextual embeddings (vector representations of words that capture meaning based on context), the *BERTScore* calculates the similarity between words in the generated and reference triples. This metric captures semantic similarity. It ranges from 0 to 1, where 1 represents a perfect semantic match. We use *BERTScore* because it can measure whether the meaning of the generated triple aligns with the gold standard, even if the wording differs.

For each of the 32 experiments (16 from D_1 and 16 from D_2), we computed the mean score across all triples for each metric.

5.4 Quantitative Analysis Results

This subsection presents our obtained results. Table 5a, Table 5b, Table 6a, and Table 6b show the results of the quantitative analysis. The first column contains the prompt type: ZS (zero-shot) or FS (few-shot). In the second column, the intents: CO (compatibility), PS (product specification), AV (product availability), and GE (general, the three previous intents together). The third and fourth columns show the results from the LLM. The fifth column shows the mean value by intent. The sixth column shows the mean by prompt specificity (the mean of specific intents (CO, PS, and AV) and the mean by GE intents). The last column shows the mean value for prompt type, ZS, and FS.

Table 5a and Table 5b show the results for D_1 (automobile) and D_2 (multidomain) with BLEU. For instance, the mean BLEU using D_2 , few-shot prompt, Sabia model and compatibility summarized text is 0.5671.

Zero-Shot vs. Few-Shot Comparison: Few-Shot training improves overall results, but its impact varies across

datasets. The intent category PS (Product Specification) suffers the most in FS, possibly due to increased complexity in handling detailed descriptions.

Llama vs. Sabiá: Llama outperforms Sabiá in most cases, especially in Zero-Shot. In D_1 ZS and D_2 ZS (automobile and multidomain zero-shot), Llama achieves a higher Mean LLM score than Sabiá. However, Llama's advantage in Few-Shot (FS) is less pronounced, with both models performing similarly in some cases (e.g., D_1 FS: 0.5018 vs. 0.4881).

Intent Types: CO (Compatibility) scores the highest across datasets and models, especially in Few-Shot. PS (Product Specification) consistently has the lowest scores, reflecting greater linguistic variability and greater difficulty in matching syntactic structures.

Question Specificity: Mean PR (Prompt Type) scores are higher in D_1 than D_2 , reinforcing that a single-domain dataset benefits more from LLMs in a structured environment. The mean IN (Intent) is more balanced across datasets but still favors D_1 .

Dataset Comparison: D_1 consistently outperforms D_2 across all settings.

Table 6a and Table 6b show the results for D_1 and D_2 with *BERTScore*. For instance, the mean *BERTScore* using D_1 , zero-shot prompt, Llama model, and compatibility summarized text is 0.8297.

Zero-Shot vs. Few-Shot Comparison: In both datasets (Table 6a and Table 6b), the few-shot (FS) configuration outperforms zero-shot (ZS), indicating that the models significantly benefit from example-based training. In D_1 , the overall *BERTScore* improves from 0.7980 to 0.9191, and in D_2 from 0.8165 to 0.9079.

Llama vs. Sabiá: In zero-shot, *Sabiá* may have a slight advantage, but in few-shot, the models converge to similar results.

Intent Types: Among the intent categories (compatibility, product specification, availability, and general – that unites the three intents), we observe some score variations. In D_1 , the most significant difference appears in the GENERAL category in ZS (0.7231 vs. 0.8274), indicating Llama struggles more with open-ended questions without additional context. In D_2 , the scores are more balanced, suggesting that a more diverse dataset helps models respond more uniformly. The compatibility intent got the best results.

Question Specificity: The average by intent (Mean IN) and by specificity (Mean SP) follows similar patterns. More specific questions (such as compatibility and product specifications) tend to achieve higher scores. More general questions tend to have lower scores, especially in zero-shot.

Dataset Comparison: D_1 (automotive) obtained better results than D_2 (multi-domain), especially in the few-shot setting. D_1 benefits more from few-shot training, likely due to its domain-specific structure. D_2 performs slightly better in zero-shot, indicating better generalization across diverse domains.

6 Discussion

The primary objective of our study was to design, develop, and evaluate a novel method for generating RDF triples

		Llama	Sabiá	Mean IN	Mean SP	Mean PR
ZS	CO	0.5926	0.6570	0.6248		
	PS	0.4726	0.4112	0.4419	0.5090	0.5265
	AV	0.4724	0.4482	0.4603		
	GE	0.5646	0.5233	0.5440	0.5440	
	Mean LLM	0.5255	0.5099			
FS	CO	0.7058	0.7472	0.7265		
	PS	0.3377	0.2765	0.3071	0.4865	0.5034
	AV	0.4400	0.4118	0.4259		
	GE	0.5236	0.5169	0.5203	0.5202	
	Mean LLM	0.5018	0.4881			

(a) BLEU results for automobile domain dataset D_1 .

		Llama	Sabiá	Mean IN	Mean SP	Mean PR
ZS	CO	0.3852	0.4190	0.4021		
	PS	0.3792	0.3421	0.3607	0.3820	0.3878
	AV	0.3871	0.3793	0.3832		
	GE	0.4089	0.3782	0.3936	0.3935	
	Mean LLM	0.3901	0.3797			
FS	CO	0.5740	0.5671	0.5706		
	PS	0.3584	0.2148	0.2866	0.4196	0.4281
	AV	0.4603	0.3432	0.4018		
	GE	0.4556	0.4175	0.4366	0.4365	
	Mean LLM	0.4621	0.3857			

(b) BLEU results for multidomain dataset D_2 . ZS stands for zero-shot and FS few-shots. CO, PS, AV, and GE stand for compatibility, product specification, availability and general, respectively. Mean IN, Mean SP and Mean PR stand for the mean by intent, by intent specificity and by prompt type. Values range from 0 to 1, with high values indicating high syntactic similarity.

from NL texts in the e-commerce context using Large Language Models. Our experimental evaluation results provided compelling evidence that our solution successfully achieved this objective. Comparing the generated triples with a gold-standard dataset in both single-store and multi-store experiments demonstrated the ability to represent compatibility relationships between products and cars accurately. The design of prompts and the use of GPT-3.5, Bloom, Llama, and Sabia models helped achieve our objective by ensuring the generation of semantically meaningful RDF triples.

Our solution incorporated key components, including an ontology defining domain-specific classes and properties, LLMs for natural language understanding, and constructed prompts to guide the LLM in the triple-generation process. Our study demonstrated its robustness in capturing compatibility relationships, and its adaptability was evident in the multi-store scenario. The use of LLMs proved effective at comprehending and translating complex NL questions into structured RDF triples, showcasing the potential of state-of-the-art LLMs for knowledge representation in e-commerce domains.

For experiment 1 (Section 4), in the single-store evaluation, a high percentage of presence for important classes and properties in the generated triples was observed, indicating a strong alignment with the gold standard dataset. The multi-store evaluation further strengthened our findings, revealing that our approach maintains consistency across diverse e-

commerce contexts. The variations in counts highlight the adaptability of our approach to different stores while consistently representing compatibility relationships. Notably, the solution demonstrated proficiency in capturing the *Full-Compatibility*, *NoCompatibility*, and *ConditionalCompatibility* classes, which are important for comprehensive compatibility assessments.

Based on the results presented in experiment 1 (Section 4), the experimental setup that produced the highest accuracy for the set of experiment 1 was E_3 , composed by $D_1 + LM_2 + P_1$, third row at the Figure 2; and for the set of experiment 2 was E_7 , composed by $D_2 + LM_2 + P_1$, seventh row at Figure 2. We found that even when varying the dataset from D_1 , with only one store, to D_2 , with multiple stores, the high accuracy of the combination of the Bloom model LM_2 and the prompt-based few-shot technique P_1 was maintained. While using few-shot techniques demonstrated promising results in our experiments, it is imperative to acknowledge that alternative approaches, such as Chain of Thoughts (CoT) [27], may yield comparable outcomes. The few-shot technique was chosen for its effectiveness in generating semantically coherent triples that align with predefined ontology rules.

For the second evaluation, described in Section 5, we seek to answer four questions: 1. How does the number of domains processed by the LLM impact the resulting triples?; 2. Does the number of examples provided in the prompt impact the resulting triples?; 3. The language specificity of the

		Llama	Sabiá	Mean IN	Mean SP	Mean PR
ZS	CO	0.8297	0.8332	0.8315		
	PS	0.8286	0.8254	0.8270	0.8207	0.7980
	AV	0.8054	0.8016	0.8035		
	GE	0.7231	0.8274	0.7753	0.7752	
	Mean LLM	0.7967	0.8219			
FS	CO	0.9592	0.9606	0.9599		
	PS	0.8862	0.8883	0.8873	0.9181	0.9191
	AV	0.9070	0.9070	0.9070		
	GE	0.9197	0.9206	0.9202	0.9201	
	Mean LLM	0.9180	0.9191			

(a) BERTScore results for automobile domain dataset D_1 .

		Llama	Sabiá	Mean IN	Mean SP	Mean PR
ZS	CO	0.8166	0.8256	0.8211		
	PS	0.8227	0.8167	0.8197	0.8145	0.8165
	AV	0.8038	0.8018	0.8028		
	GE	0.8193	0.8175	0.8184	0.8184	
	Mean LLM	0.8156	0.8154			
FS	CO	0.9433	0.9440	0.9437		
	PS	0.8859	0.8718	0.8789	0.9088	0.9079
	AV	0.9118	0.8960	0.9039		
	GE	0.9106	0.9035	0.9071	0.9070	
	Mean LLM	0.9129	0.9038			

(b) BERTScore results for multidomain dataset D_2 . ZS stands for zero-shot and FS few-shots. CO, PS, AV, and GE stand for compatibility, product specification, availability and general, respectively. Mean IN, Mean SP and Mean PR stand for the mean by intent, by intent specificity and by prompt type. Values range from 0 to 1, with high values indicating high semantic similarity.

LLM impacts the resulting triples?; 4. Does the variability of intents impact the resulting triples?

To answer the impact of the number of domains (one vs. many), the results showed that the models perform better in single-domain (D_1) than in multi-domain (D_2) across both BLEU and BERTScore. This suggests that working within a single domain helps the models generate more lexically and semantically aligned triples.

To answer the impact of the number of examples (zero-shot vs. few-shot), Few-shot prompting improves results in the majority of cases.

To assess the impact of language specificity (multilingual vs. Portuguese fine-tuned), the results indicate that Sabiá outperforms Llama on D_1 but underperforms on D_2 across both BLEU and BERTScore. This means that a language-specific model excels in structured, domain-focused tasks but struggles with adaptability in multi-domain settings.

To assess the impact of intent variability (one intent vs. three), the results show that BLEU and BERTScore scores are lower for datasets with multiple intents, indicating that increased intent diversity introduces greater linguistic variation, making it harder for models to generate syntactically and semantically aligned triples. This suggests that models struggle with intent generalization.

Our primary research question (cf. Section 1) was how to generate triples using LLMs based on ontology specifications. Based on our method, evaluation, and results, we orig-

inally found that it is possible to incorporate LLMs into the triple-generation process by embedding ontology statements in the prompt. We characterize this as the main finding of this section and the study's key originality.

Even though our method generated triples that were quantitatively and qualitatively similar to the gold standard, we faced challenges during its planning and execution. Among these challenges, we mention:

LLM window size: One notable challenge in our method is the potential impact of the window size of the LLMs. As the window size of LLMs evolves, capturing more extensive context, the prompts generated by our method vary in size. This variation is influenced by factors such as the ontology's list of essential classes and properties and the number of examples in the few-shot section of the prompt. When dealing with substantial ontologies, there is a risk that the LLM might struggle to maintain coherence within the provided context.

Variability of properties and classes in Few Shots: An extension of the window size challenge lies in the variability of properties and classes within the few-shot section of the prompt. Whereas our example successfully incorporated examples for *FullCompatibility*, *ConditionalCompatibility*, and *NoCompatibility*, the question arises regarding scalability when dealing with numerous classes. The issue is magnified when attempting to cover a broad range of classes within a limited few-shot context (e.g., if we had hundreds of classes). Addressing this aspect may involve dynamically

adapting the few-shot examples to the ontology's complexity, ensuring effective knowledge transfer to the LLM while maintaining coherence.

Enhancing prompting description: The overall efficacy of the combination of P_1 and D_1 in the evaluation could be further refined by enhancing the prompting description. Instances of generating incorrect triples were infrequent; most such cases were linked to properties like compatibility and fabrication year. Strengthening the prompting description, particularly for these specific cases, offers a potential avenue for improvement. By providing more explicit instructions or refining the examples within the prompt, we aim to minimize discrepancies and enhance the accuracy of the generated RDF triples.

The successful application of our solution has promising implications for knowledge representation in e-commerce. The ability to accurately generate RDF triples from human NL questions opens the door to updating KGs and supporting more advanced decision-making systems within e-commerce platforms.

One avenue for future research is to vary the dataset's domain. While we demonstrated the results of our approach in the automotive domain, it is valid to explore its effectiveness across diverse e-commerce domains, such as cellphones, furniture, and other retail sectors. Understanding how well the method generalizes across different product categories may provide valuable insights into its versatility and potential adaptations needed for specific domains.

Another direction for future work lies in automating specific manual steps within our framework. Specifically, exploring techniques to extract all relevant classes and properties from an ontology is a key area for improvement. Currently, these elements are manually curated, and automating this process would enhance the scalability and efficiency of our proposal. Investigating machine learning and NL processing methods for ontology extraction could streamline the knowledge acquisition phase, reduce reliance on manual effort, and open avenues for broader ontology adoption.

Additionally, to enhance the accessibility and user-friendliness of our outcomes, a future endeavor could involve designing and developing a visual interface. Such an interface would empower domain experts to contribute to the KG by intuitively adding triples based on textual information. This interface could bridge NL queries and the KGs, allowing experts to enrich the underlying ontology. A well-designed visual software tool could facilitate the inclusion of additional knowledge, ensuring that the solution remains user-friendly and accessible to a broader audience beyond experts in ontology management.

7 Conclusion

In e-commerce, the essential exchange of information between humans and machines faces a challenge in bridging the gap between human-friendly language and machine-understandable data structures. The primary challenge is automating the generation of RDF triples from natural-language questions in e-commerce to connect them to a structured ontology. Complexities arise from territory-specific

expressions, ambiguous terms, and context-dependent references, hindering precise information extraction. This investigation established a robust automatic RDF triple-generation method that successfully addressed the challenges posed by NL questions in the e-commerce domain. We described and evaluated how we used LLMs to populate existing KGs with RDF triples, varying the LLM and prompt style across datasets. Our main contribution emphasized the method for incorporating unstructured elements (texts in NL) and structured elements (ontology elements) into a well-defined prompt to leverage the generation of RDF triples consistent with ontological structures. The findings not only validate the effectiveness of our approach but also pave the way for further advancements in utilizing language models for structured knowledge representation in complex domains.

Declarations

Authors' Contributions

AGR: Conceptualization, Data curation, Investigation, Methodology, Validation, Visualization, Writing. JCR: Supervision, Writing-review & editing, Funding acquisition, Project administration. All authors read and approved the final manuscript.

Competing interests

The authors declare that they have no competing interests.

Acknowledgements

THIS DECLARATION IS OPTIONAL. This is a multiline text of acknowledgments. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

Funding

This study was financed by the National Council for Scientific and Technological Development - Brazil (CNPq) process number 140213/2021-0. In addition, this research was partially funded by the São Paulo Research Foundation (FAPESP) (grants #2022/13694-0, #2022/15816-5 and #2024/07716-6). The opinions expressed in this work do not necessarily reflect those of the funding agencies. We thank GoBots for providing the infrastructure used in this research.

Availability of data and materials

The datasets generated and/or analysed during the current study are available in <https://doi.org/10.25824/redu/RRGTKC>.

References

- [1] Abonizio, H., Almeida, T. S., Laitz, T., Junior, R. M., Bonás, G. K., Nogueira, R., and Pires, R. (2024). Sabi\'a-3 technical report. *arXiv preprint arXiv:2410.12049*.

- [2] Arnaout, H. and Elbassuoni, S. (2018). Effective searching of rdf knowledge graphs. *Journal of Web Semantics*, 48:66–84. DOI: 10.1016/j.websem.2017.12.001.
- [3] Auer, S., Bizer, C., Kobilarov, G., Lehmann, J., Cyganiak, R., and Ives, Z. (2007). Dbpedia: A nucleus for a web of open data. In *international semantic web conference*, pages 722–735. Springer.
- [4] Balabin, H., Hoyt, C. T., Birkenbihl, C., Gyori, B. M., Bachman, J., Kodamullil, A. T., Plöger, P. G., Hofmann-Apitius, M., and Domingo-Fernández, D. (2022). Stonkgs: a sophisticated transformer trained on biomedical text and knowledge graphs. *Bioinformatics*, 38(6):1648–1656.
- [5] Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901.
- [6] Bytyci, A., Ramosaj, L., and Bytyci, E. (2023). Review of automatic and semi-automatic creation of knowledge graphs from structured and unstructured data. In *RTA-CSIT*, pages 72–79.
- [7] Ehrlinger, L. and Wöß, W. (2016). Towards a definition of knowledge graphs. *SEMANTiCS (Posters, Demos, SuCCESS)*, 48(1-4):2.
- [8] Grimm, S. (2009). Knowledge representation and ontologies. In *Scientific data mining and knowledge discovery: principles and foundations*, pages 111–137. Springer.
- [9] Grover, A. and Leskovec, J. (2016). node2vec: Scalable feature learning for networks. In *Proceedings of the 22nd ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 855–864.
- [Gruber] Gruber, T. What is an ontology.
- [11] Gruber, T. R. (1995). Toward principles for the design of ontologies used for knowledge sharing. *International Journal of Human-Computer Studies*, 43(5):907 – 928. DOI: <https://doi.org/10.1006/ijhc.1995.1081>.
- [12] Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., and Kang, J. (2020). Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, 36(4):1234–1240.
- [13] Li, F.-L., Chen, H., Xu, G., Qiu, T., Ji, F., Zhang, J., and Chen, H. (2020). Alimekg: Domain knowledge graph construction and application in e-commerce. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, pages 2581–2588.
- [14] Liu, Y., Zhang, T., Liang, Z., Ji, H., and McGuinness, D. L. (2018). Seq2rdf: An end-to-end application for deriving triples from natural language text. In *CEUR Workshop Proceedings*, volume 2180. CEUR-WS.
- [15] Ora, L. (1999). Resource description framework (rdf) model and syntax specification. <http://www.w3.org/TR/REC-rdf-syntax/>.
- [16] Papineni, K., Roukos, S., Ward, T., and Zhu, W.-J. (2002). Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318.
- [17] Regino, A. and dos Reis, J. C. (2025). Can llms be knowledge graph curators for validating triple insertions? In Genet Asefa Gesese, Harald Sack, H. P. A. M.-P. and Chen, L., editors, *Proceedings of the Workshop on Generative AI and Knowledge Graphs (GenAIK) co-located with the 31st International Conference on Computational Linguistics (COLING 2025)*, pages 87–99, Abu Dhabi, UAE. International Committee on Computational Linguistics.
- [18] Regino, A. G., Caus, R. O., Hochgreb, V., and dos Reis, J. C. (2023a). From natural language texts to rdf triples: A novel approach to generating e-commerce knowledge graphs. In Coenen, F., Fred, A., Aveiro, D., Dietz, J., Bernardino, J., Masciari, E., and Filipe, J., editors, *Knowledge Discovery, Knowledge Engineering and Knowledge Management*, pages 149–174. Springer Nature Switzerland. DOI: 10.1007/978-3-031-43471-6_7.
- [19] Regino, A. G., Caus, R. O., Hochgreb, V., and Reis, J. C. d. (2023b). Leveraging knowledge graphs for e-commerce product recommendations. *SN Computer Science*, 4(5):689.
- [20] Rossanez, A., Dos Reis, J. C., Torres, R. d. S., and de Ribaupierre, H. (2020). Kgen: a knowledge graph generator from biomedical scientific literature. *BMC medical informatics and decision making*, 20(4):1–24.
- [21] Sant’Anna, D. T., Caus, R. O., dos Santos Ramos, L., Hochgreb, V., and dos Reis, J. C. (2020). Generating knowledge graphs from unstructured texts: Experiences in the e-commerce field for question answering. In *Advances in Semantics and Linked Data: Joint Workshop Proceedings from ISWC 2020*, pages 56–71.
- [22] Scao, T. L., Fan, A., Akiki, C., Pavlick, E., Ilić, S., Hesslow, D., Castagné, R., Luccioni, A. S., Yvon, F., Gallé, M., et al. (2022). Bloom: A 176b-parameter open-access multilingual language model. *arXiv preprint arXiv:2211.05100*.
- [23] Schneider, P., Schopf, T., Vladika, J., Galkin, M., Simperl, E., and Matthes, F. (2022). A decade of knowledge graphs in natural language processing: A survey. *arXiv preprint arXiv:2210.00105*.
- [24] Suchanek, F. and Weikum, G. (2013). Knowledge harvesting from text and web sources. In *2013 IEEE 29th International Conference on Data Engineering (ICDE)*, pages 1250–1253. DOI: 10.1109/ICDE.2013.6544916.
- [25] Tang, J., Xu, C., and Zhang, W. (2023). Construction and accurate retrieval method of knowledge graph of automobile engine fault. In *2023 IEEE 2nd International Conference on Electrical Engineering, Big Data and Algorithms (EEBDA)*, pages 336–345. IEEE.
- [26] Wang, L., Hao, H., Yan, X., Zhou, T. H., and Ryu, K. H. (2025). From biomedical knowledge graph construction to semantic querying: a comprehensive approach. *Scientific Reports*, 15(1):8523.
- [27] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Xia, F., Chi, E., Le, Q. V., Zhou, D., et al. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837.
- [28] Wu, X., Tang, Y., Zhou, C., Zhu, G., Song, J., and Liu, G. (2022). An intelligent search engine based on knowledge graph for power equipment management. In *2022 5th Inter-*

national Conference on Energy, Electrical and Power Engineering (CEEPE), pages 370–374. IEEE.

- [29] Yasunaga, M., Ren, H., Bosselut, A., Liang, P., and Leskovec, J. (2021). QA-GNN: Reasoning with language models and knowledge graphs for question answering. In Toutanova, K., Rumshisky, A., Zettlemoyer, L., Hakkani-Tur, D., Beltagy, I., Bethard, S., Cotterell, R., Chakraborty, T., and Zhou, Y., editors, *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 535–546, Online. Association for Computational Linguistics. DOI: 10.18653/v1/2021.naacl-main.45.
- [30] Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., and Artzi, Y. (2019). Bertscore: Evaluating text generation with bert. *arXiv preprint arXiv:1904.09675*.